

绘图组件使用文档

本文档示例基于第一问清洗数据与特征数据生成。调用前建议先添加项目 `src` 路径。

```
import sys
import numpy as np
import pandas as pd

sys.path.insert(0, r"D:\8\Desktop\CMathc\src")

from utils.plot_utils import *
from utils.color_palettes import get_sci_height_colors, get_sci_deep_colors
```

组件目录

基础图	
— plot_line	折线图
— plot_scatter	散点图
— plot_bar	柱状图
— plot_hist	直方图
— plot_box	箱线图
— plot_kde	核密度图
— plot_joint_hist	联合直方图
模型诊断	
— plot_residual	残差图
— plot_qq	Q-Q 图
— plot_corr_heatmap	相关性热力图
不确定性	
— plot_band	置信区间 / 波动区间带图
空间与场	
— plot_profile	垂直廓线图
— plot_section	时间-高度剖面图
— plot_heatmap	热力图
— plot_contour	等高线图
— plot_bubble	气泡图
— plot_wind_polar	风向风速极坐标图
— plot_3d_scatter	三维散点图
— plot_3d_surface	三维曲面图
— plot_3d_wireframe	三维网格图
— plot_gis_layers	GIS 多图层组合图
— plot_map_heatmap	地图热力图
优化	
— plot_pareto	Pareto 前沿图

说明：

- `plot_line` 可覆盖时间序列、训练损失、预测值与真实值对比、PSO 收敛曲线等普通折线类结果图。
- `plot_scatter`、`plot_hist`、`plot_kde`、`plot_joint_hist` 主要用于数据探索与变量关系分析。
- `plot_residual`、`plot_qq`、`plot_corr_heatmap` 主要用于模型诊断和变量相关性分析。
- `plot_profile`、`plot_section`、`plot_contour`、三维图及 GIS 组件主要用于空间、气象、物理场类问题。
- `plot_pareto` 用于双目标优化结果展示。

数学建模题型与推荐图

题型 / 任务	主要目的	推荐组件	常见论文用途
数据探索与描述统计	看分布、异常值、变量关系	<code>plot_hist</code> 、 <code>plot_box</code> 、 <code>plot_kde</code> 、 <code>plot_scatter</code> 、 <code>plot_joint_hist</code>	数据预处理、分布特征、异常点说明
相关性与特征分析	判断变量之间的线性或非线性关系	<code>plot_corr_heatmap</code> 、 <code>plot_scatter</code> 、 <code>plot_bubble</code>	特征筛选、解释变量关系
回归 / 拟合	比较真实值与拟合结果、检查误差	<code>plot_line</code> 、 <code>plot_scatter</code> 、 <code>plot_residual</code> 、 <code>plot_qq</code>	拟合效果、残差诊断、模型合理性
时间序列 / 预测	展示趋势、训练过程、预测结果	<code>plot_line</code> 、 <code>plot_band</code>	时间趋势、LSTM 损失、预测值与实际值、预测区间
分类问题	比较不同类别指标或结果分布	<code>plot_bar</code> 、 <code>plot_hist</code> 、 <code>plot_scatter</code>	类别差异、分类结果统计；若正式做分类，可后续补混淆矩阵 / ROC
聚类问题	展示样本分组和聚类结构	<code>plot_scatter</code> 、 <code>plot_3d_scatter</code> 、 <code>plot_heatmap</code>	聚类结果、不同类别空间分布
综合评价 / 方案比较	比较多个方案或模型的指标	<code>plot_bar</code> 、 <code>plot_line</code> 、 <code>plot_heatmap</code>	模型对比、方案优劣、指标变化
单目标优化	展示目标函数或迭代收敛过程	<code>plot_line</code> 、 <code>plot_scatter</code>	PSO / GA / SA 等算法收敛曲线
多目标优化	展示多个目标之间的权衡	<code>plot_pareto</code>	Pareto 前沿、成本-风险、收益-能耗权衡
灵敏度 / 鲁棒性分析	观察参数变化对输出的影响	<code>plot_line</code> 、 <code>plot_band</code> 、 <code>plot_bar</code>	参数敏感性、均值 ± 波动范围
空间插值 / 地理问题	展示空间位置、区域风险与空间分布	<code>plot_gis_layers</code> 、 <code>plot_map_heatmap</code> 、 <code>plot_contour</code> 、 <code>plot_scatter</code>	GIS 底图、空间风险场、站点与区域分布

题型 / 任务	主要目的	推荐组件	常见论文用途
气象 / 环境 / 垂直结构	展示变量随高度、时间和空间的变化	<code>plot_profile</code> 、 <code>plot_section</code> 、 <code>plot_wind_polar</code> 、 <code>plot_heatmap</code>	温度廓线、风场、时间-高度剖面
物理场 / PDE / 连续空间模型	展示二维或三维连续场	<code>plot_contour</code> 、 <code>plot_3d_surface</code> 、 <code>plot_3d_wireframe</code> 、 <code>plot_3d_scatter</code>	温度场、压力场、流场、风险场
不确定性分析	展示置信区间、标准差或预测区间	<code>plot_band</code> 、 <code>plot_box</code> 、 <code>plot_kde</code>	均值 ± 标准差、预测区间、结果稳定性

表中优先列出当前已经封装好的组件。实际比赛中不需要每种图都画，优先选择能够直接支撑模型结论的图。

基础图

折线图

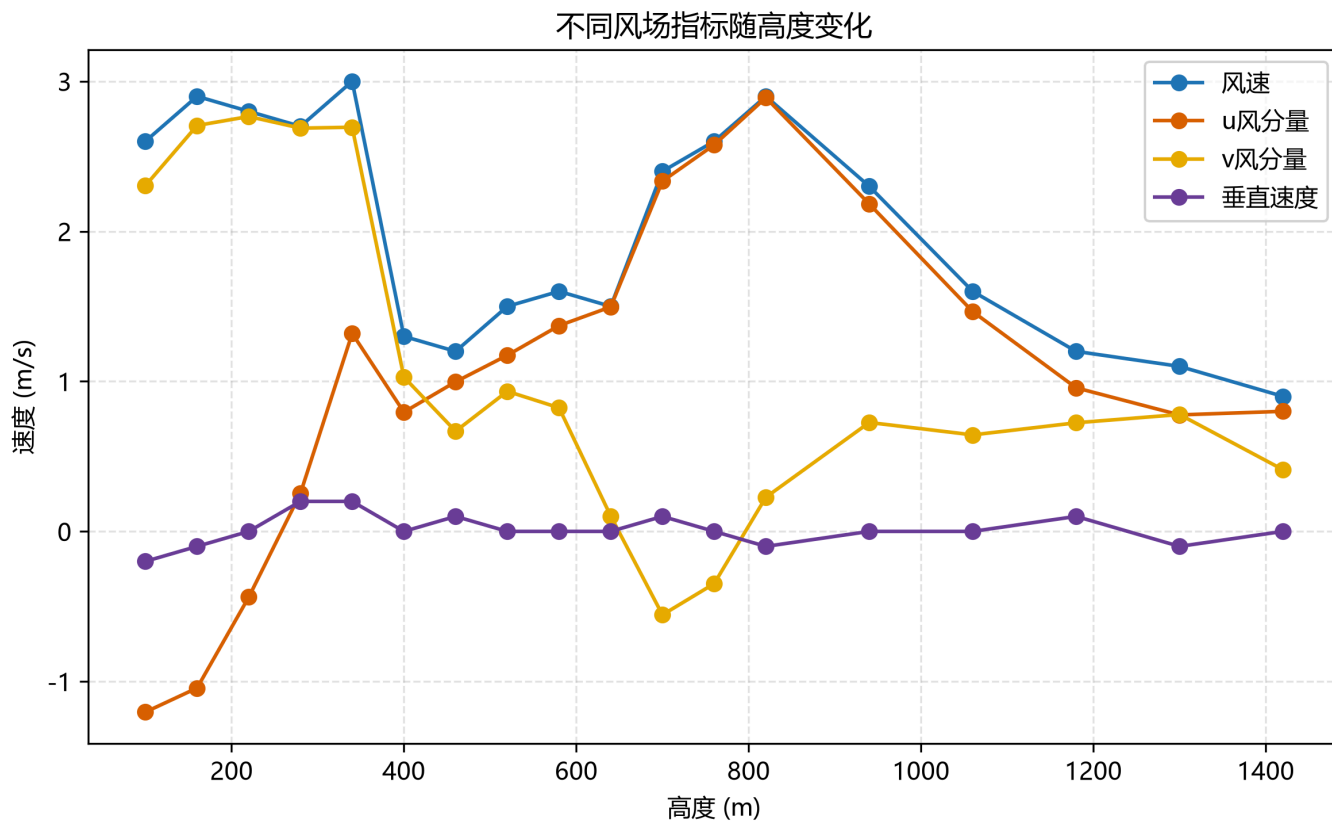
`plot_line` 可用于普通时间序列、多个指标对比、训练损失曲线、预测值与实际值对比，以及按高度或类别分组的多条曲线。

多指标折线图

```
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")
df["time"] = pd.to_datetime(df["time"])

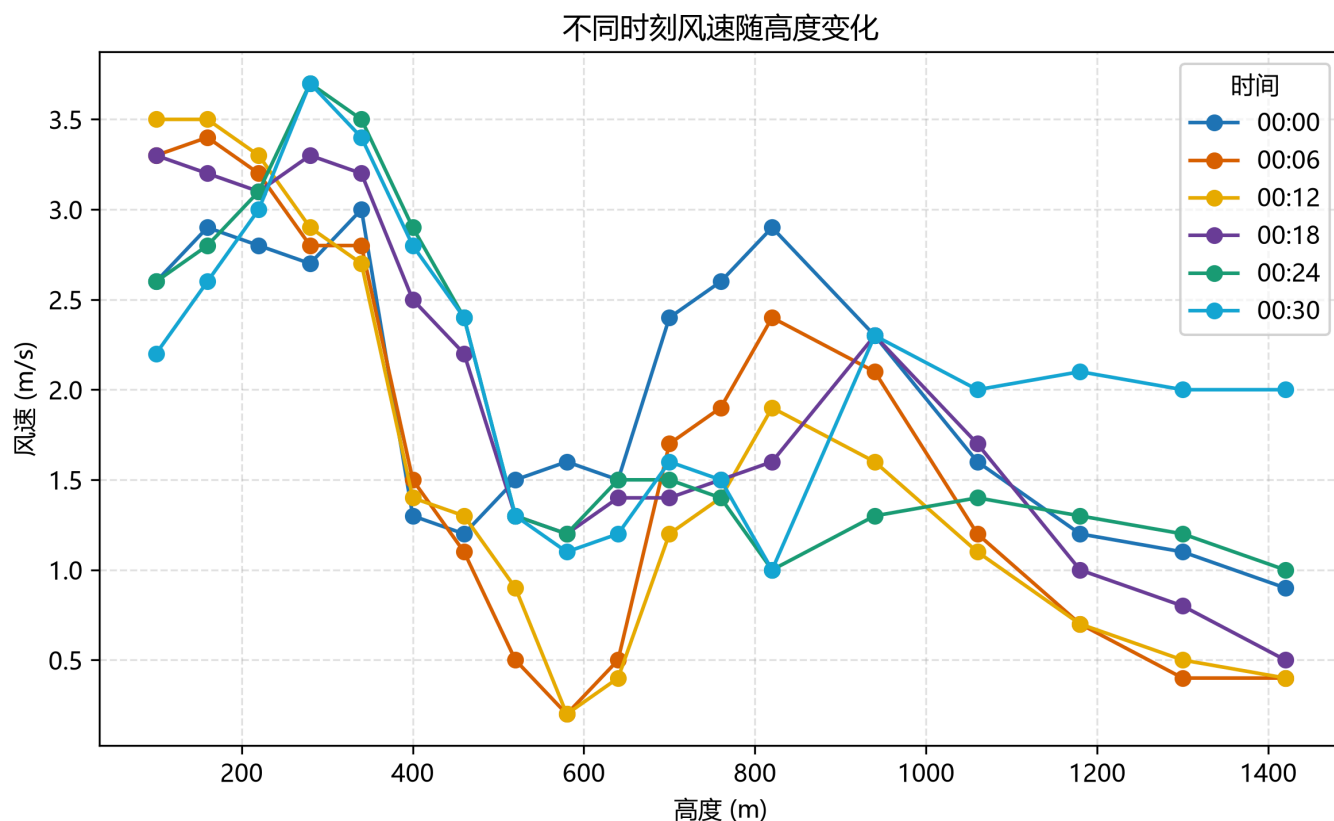
df_plot = df[
    (df["station"] == "a")
    & (df["time"] == df["time"].min())
    & (df["height"] >= 100)
    & (df["height"] <= 1500)
].copy()

plot_line(
    df_plot,
    x="height",
    y=["wind_speed", "u", "v", "vertical_velocity"],
    labels=["风速", "u风分量", "v风分量", "垂直速度"],
    title="不同风场指标随高度变化",
    xlabel="高度 (m)",
    ylabel="速度 (m/s)",
    colors=get_sci_deep_colors(4),
    linestyle=["-", "--", "-.", ":"],
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\折线图-多指标.png",
)
```



分组折线图

```
df =  
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")  
df["time"] = pd.to_datetime(df["time"])  
df["time_label"] = df["time"].dt.strftime("%H:%M")  
df_plot = df[(df["station"] == "a") & (df["height"] >= 100) & (df["height"] <=  
1000)].copy()  
  
df_plot["height_group"] = (df_plot["height"] // 100).astype(int)  
  
plot_line(  
    df_plot,  
    x="time_label",  
    y="relative_humidity",  
    group="height_group",  
    legend="高度层",  
    title="不同高度层相对湿度时间变化",  
    xlabel="时间",  
    ylabel="相对湿度 (%)",  
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\折线图-湿度时间变化.png",  
)
```



对于 LSTM 损失、预测值与实际值对比、PSO 收敛曲线，只需要更换 x 和 y ：

```
plot_line(history, x="epoch", y="loss", xlabel="Epoch", ylabel="MSE损失")

plot_line(
    result,
    x="index",
    y=["actual", "predicted"],
    labels=["实际值", "预测值"],
    linestyle=["-", "--"],
    xlabel="样本索引",
    ylabel="TKE值",
)

plot_line(pso, x="iteration", y="best_cost", xlabel="迭代次数", ylabel="最优代价J")
```

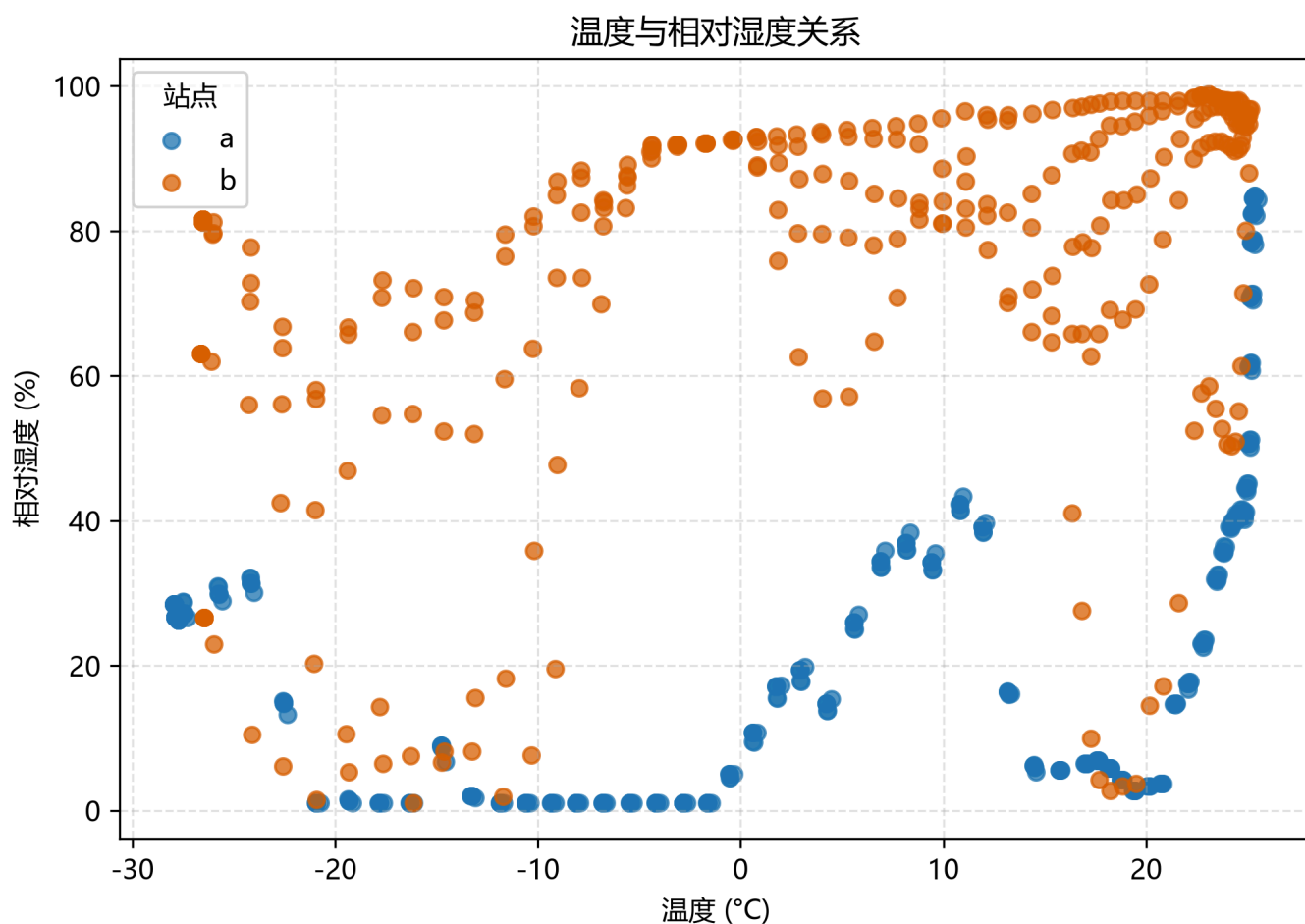
散点图

`plot_scatter` 用于观察两个变量之间的关系，也可以通过 `group` 按类别分别绘制。

```
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")

plot_scatter(
    df,
    x="temperature",
```

```
y="relative_humidity",
group="station",
legend="站点",
title="温度与相对湿度关系",
xlabel="温度 (°C)",
ylabel="相对湿度 (%)",
colors=get_sci_deep_colors(df["station"].nunique()),
save_path=r"D:\8\Desktop\CMathc\src\utils\output\散点图-温度与湿度关系.png",
)
```



柱状图

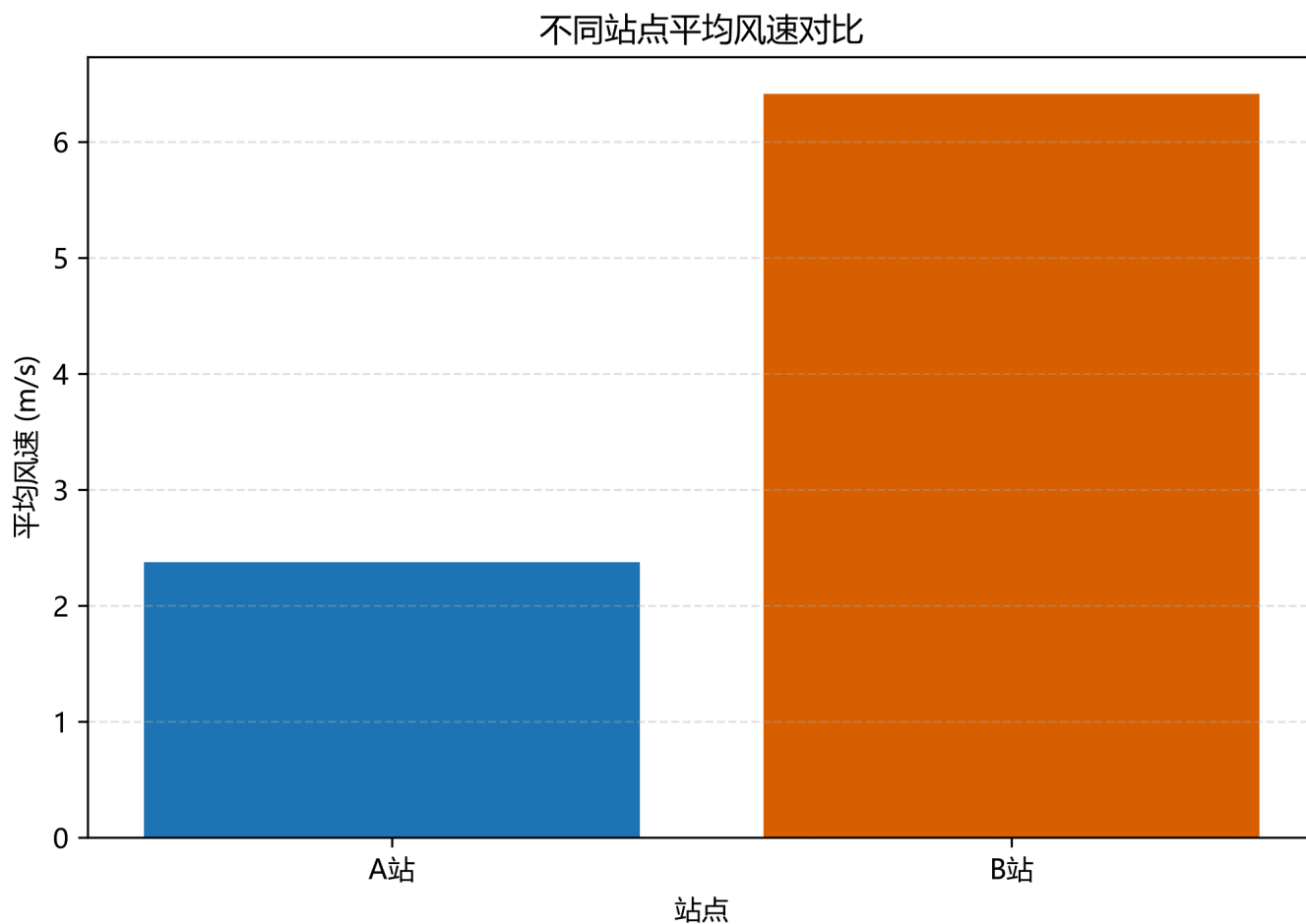
`plot_bar` 适合比较不同站点、模型或方案的统计指标。下面先按站点计算平均风速，再调用绘图组件。

```
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")

bar_data = df.groupby("station", as_index=False)["wind_speed"].mean()

plot_bar(
    bar_data,
    x="station",
    y="wind_speed",
    labels=["A站", "B站"],
    title="不同站点平均风速对比",
)
```

```
xlabel="站点",
ylabel="平均风速 (m/s)",
colors=get_sci_deep_colors(len(bar_data)),
save_path=r"D:\8\Desktop\CMathc\src\utils\output\柱状图-站点平均风速.png",
)
```

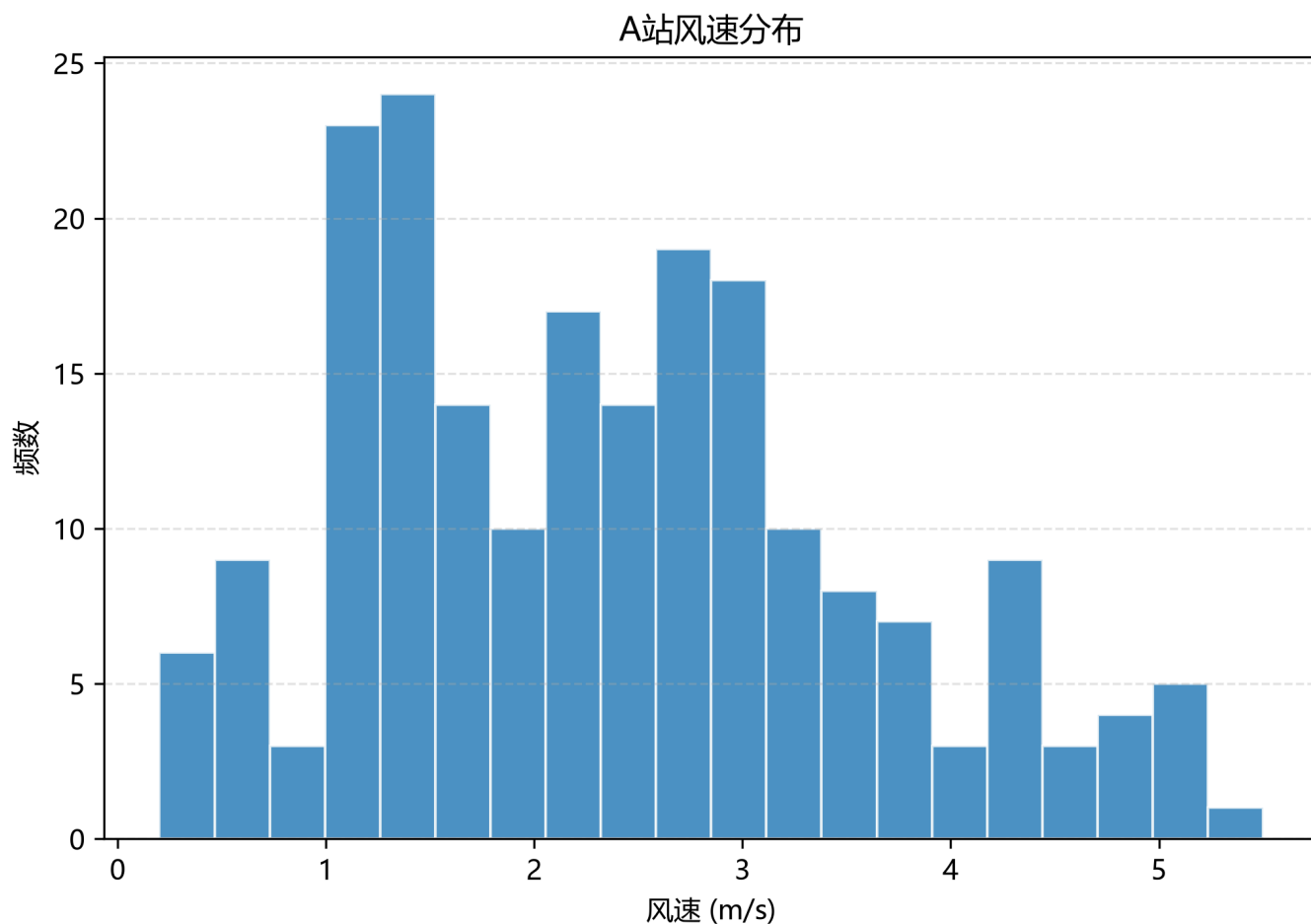


直方图

`plot_hist` 用于查看单个变量的分布。下面绘制 A 站全部观测的风速分布。

```
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")
df_a = df[df["station"] == "a"].copy()

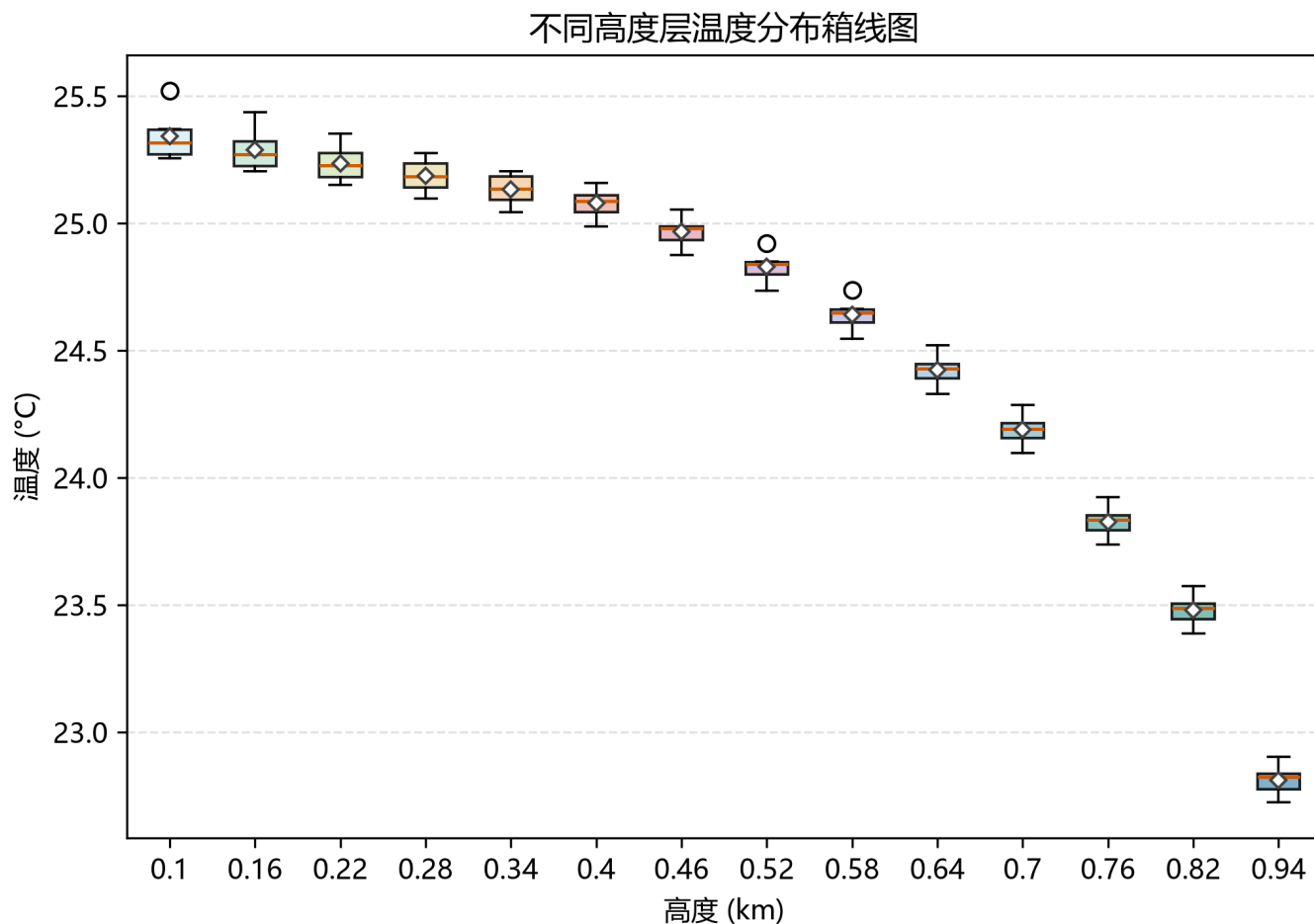
plot_hist(
    df_a,
    value="wind_speed",
    bins=20,
    title="A站风速分布",
    xlabel="风速 (m/s)",
    ylabel="频数",
    colors=get_sci_deep_colors(1),
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\直方图-A站风速分布.png",
)
```



箱线图

```
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")
df_plot = df[(df["station"] == "a") & (df["height"] >= 100) & (df["height"] <=
940)].copy()
df_plot["height_km"] = (df_plot["height"] / 1000).round(2)
heights = sorted(df_plot["height_km"].dropna().unique())

plot_box(
    df_plot,
    value="temperature",
    group="height_km",
    legend="高度 (km)",
    title="不同高度层温度分布箱线图",
    ylabel="温度 (°C)",
    labels=[f"{h:g}" for h in heights],
    colors=get_sci_height_colors(len(heights)),
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\箱线图-不同高度层温度分布.png",
)
```



核密度图

`plot_kde` 用平滑密度曲线比较变量的分布形态。它不是每道题都必须使用，但当需要比较不同站点、不同高度层或不同方案的分布差异时，比普通直方图更直观。

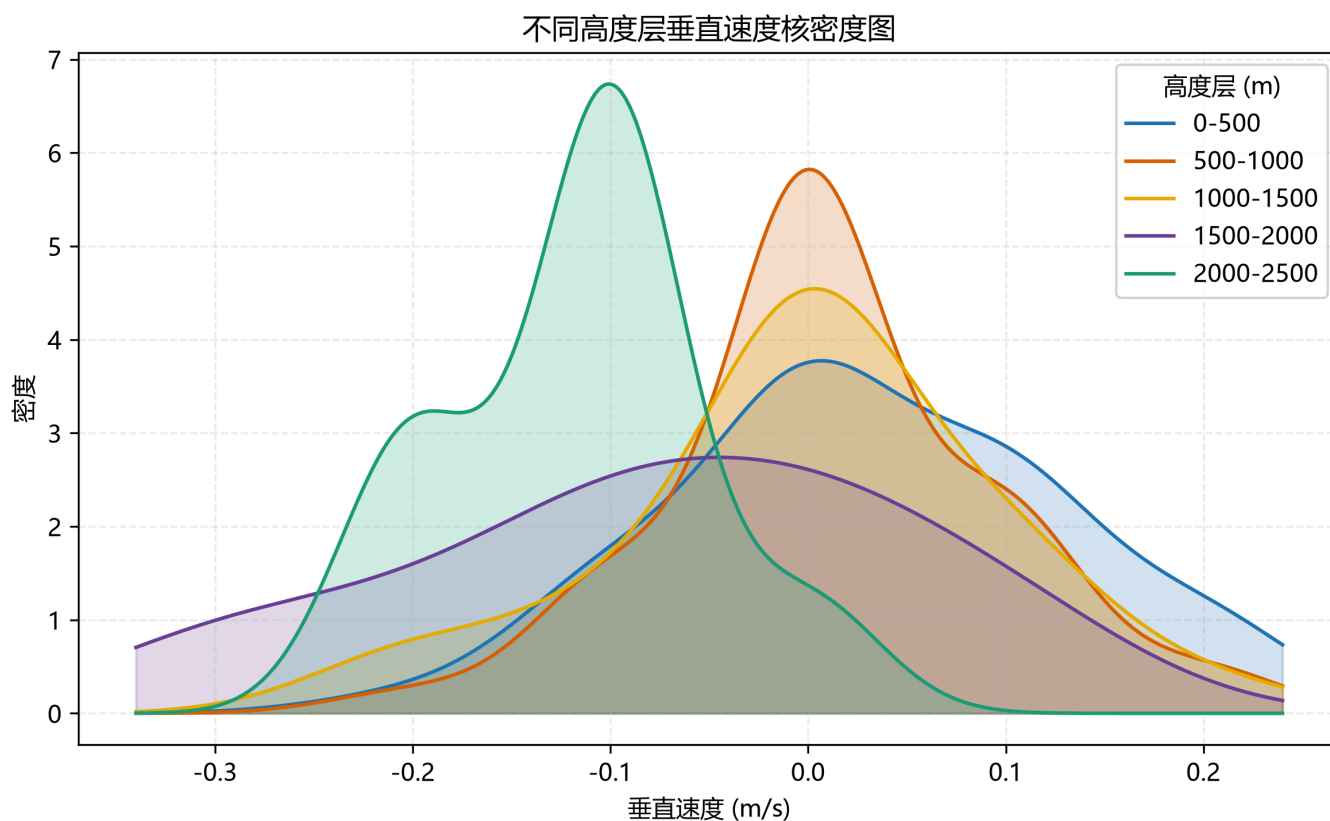
下面把 A 站 2500 m 以下划分为 5 个高度层，比较垂直速度的分布：

```
df =  
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")  
  
df_plot = df[  
    (df["station"] == "a")  
    & (df["height"] <= 2500)  
].copy()  
  
df_plot["height_group"] = pd.cut(  
    df_plot["height"],  
    bins=[0, 500, 1000, 1500, 2000, 2500],  
    labels=["0-500", "500-1000", "1000-1500", "1500-2000", "2000-2500"],  
)  
  
plot_kde(  
    df_plot,  
    value="vertical_velocity",  
    group="height_group",  
    legend="高度层 (m)",
```

```

title="不同高度层垂直速度核密度图",
xlabel="垂直速度 (m/s)",
ylabel="密度",
colors=get_sci_deep_colors(5),
save_path=r"D:\8\Desktop\CMathc\src\utils\output\核密度图-不同高度层垂直速度.png",
)

```



联合直方图

`plot_joint_hist` 同时展示两个连续变量的二维联合分布，以及横轴、纵轴变量各自的边缘直方图。适合观察两个变量之间的关系和样本密集区域。

```

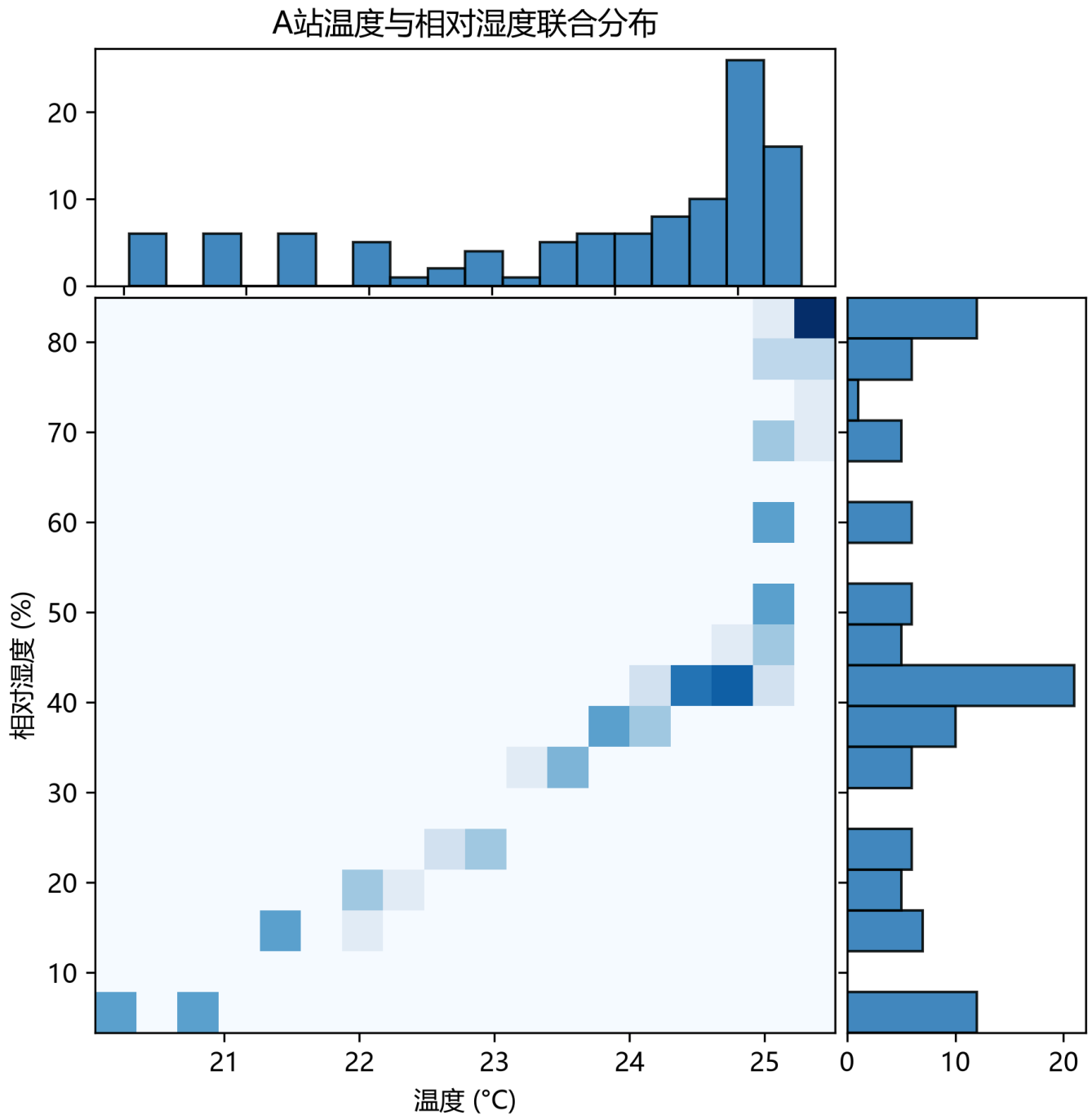
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")

df_plot = df[
    (df["station"] == "a")
    & (df["height"] <= 1500)
].copy()

plot_joint_hist(
    df_plot,
    x="temperature",
    y="relative_humidity",
    bins=18,
    title="A站温度与相对湿度联合分布",
    xlabel="温度 (°C)",

```

```
ylabel="相对湿度 (%)",  
cmap="Blues",  
colors=get_sci_deep_colors(1),  
save_path=r"D:\8\Desktop\CMathc\src\utils\output\联合直方图-温度与湿度.png",  
)
```



模型诊断

残差图

`plot_residual` 接收真实值列和预测值列，并绘制

$$e = y - \hat{y}$$

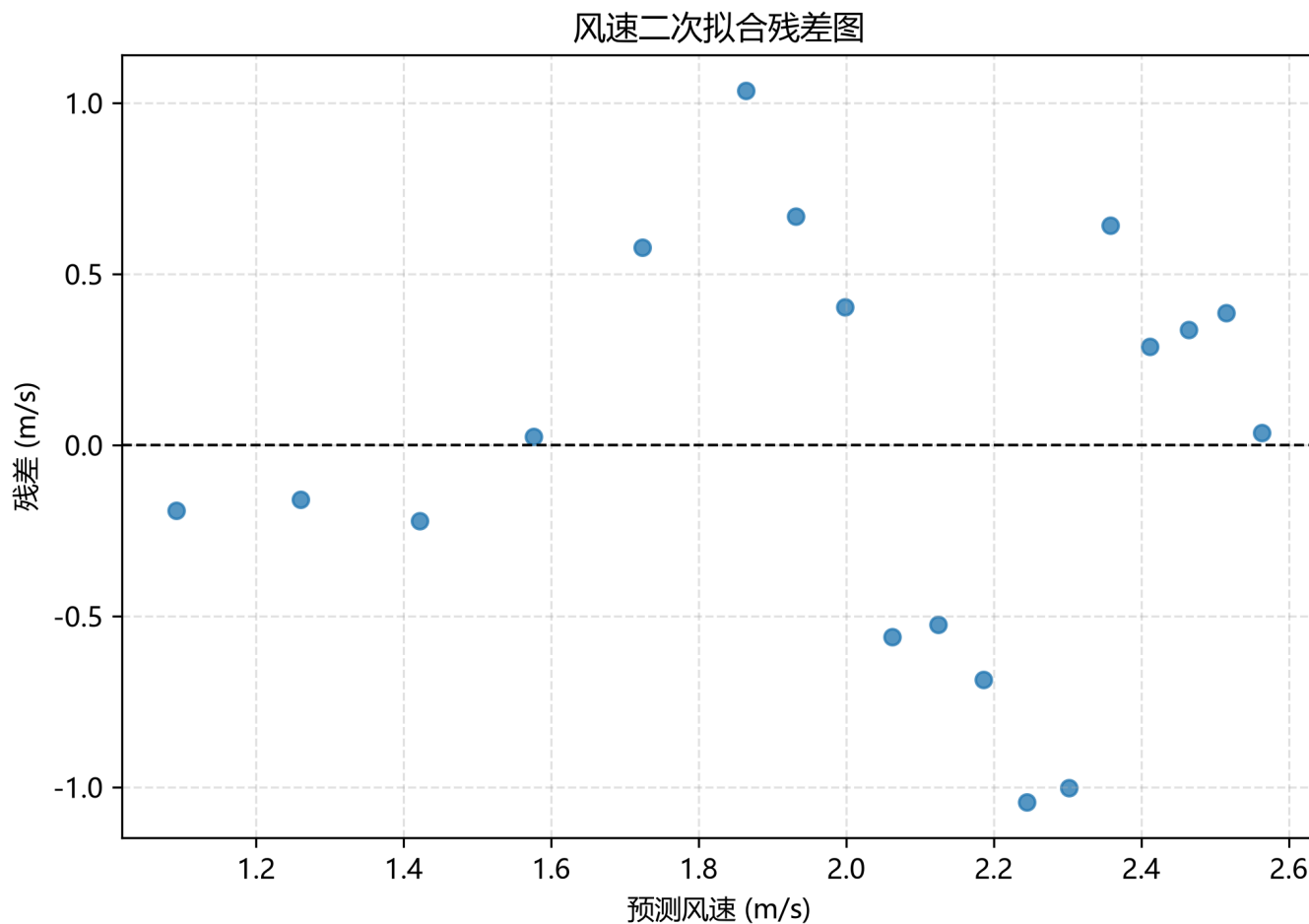
相对于预测值的分布。现有特征数据还没有正式模型预测值，因此下面仅用某一时刻的风速-高度二次拟合来验证组件。正式建模后，把 **actual** 和 **predicted** 换成模型真实值列与预测值列即可。

```
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")
df["time"] = pd.to_datetime(df["time"])

fit_data = df[
    (df["station"] == "a")
    & (df["time"] == df["time"].min())
    & (df["height"] <= 1500)
][["height", "wind_speed"]].dropna().copy()

coef = np.polyfit(fit_data["height"], fit_data["wind_speed"], 2)
fit_data["predicted_wind_speed"] = np.polyval(coef, fit_data["height"])

plot_residual(
    fit_data,
    actual="wind_speed",
    predicted="predicted_wind_speed",
    title="风速二次拟合残差图",
    xlabel="预测风速 (m/s)",
    ylabel="残差 (m/s)",
    colors=get_sci_deep_colors(1),
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\残差图-风速二次拟合.png",
)
```



Q-Q图

`plot_qq` 用于比较样本分位数与理论正态分位数。回归模型中常用于检查残差是否近似正态分布。下面继续使用风速-高度二次拟合得到的残差进行演示：

```
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")
df["time"] = pd.to_datetime(df["time"])

fit_data = df[
    (df["station"] == "a")
    & (df["time"] == df["time"].min())
    & (df["height"] <= 1500)
][["height", "wind_speed"]].dropna().copy()

coef = np.polyfit(fit_data["height"], fit_data["wind_speed"], 2)
fit_data["predicted_wind_speed"] = np.polyval(coef, fit_data["height"])
fit_data["residual"] = fit_data["wind_speed"] - fit_data["predicted_wind_speed"]

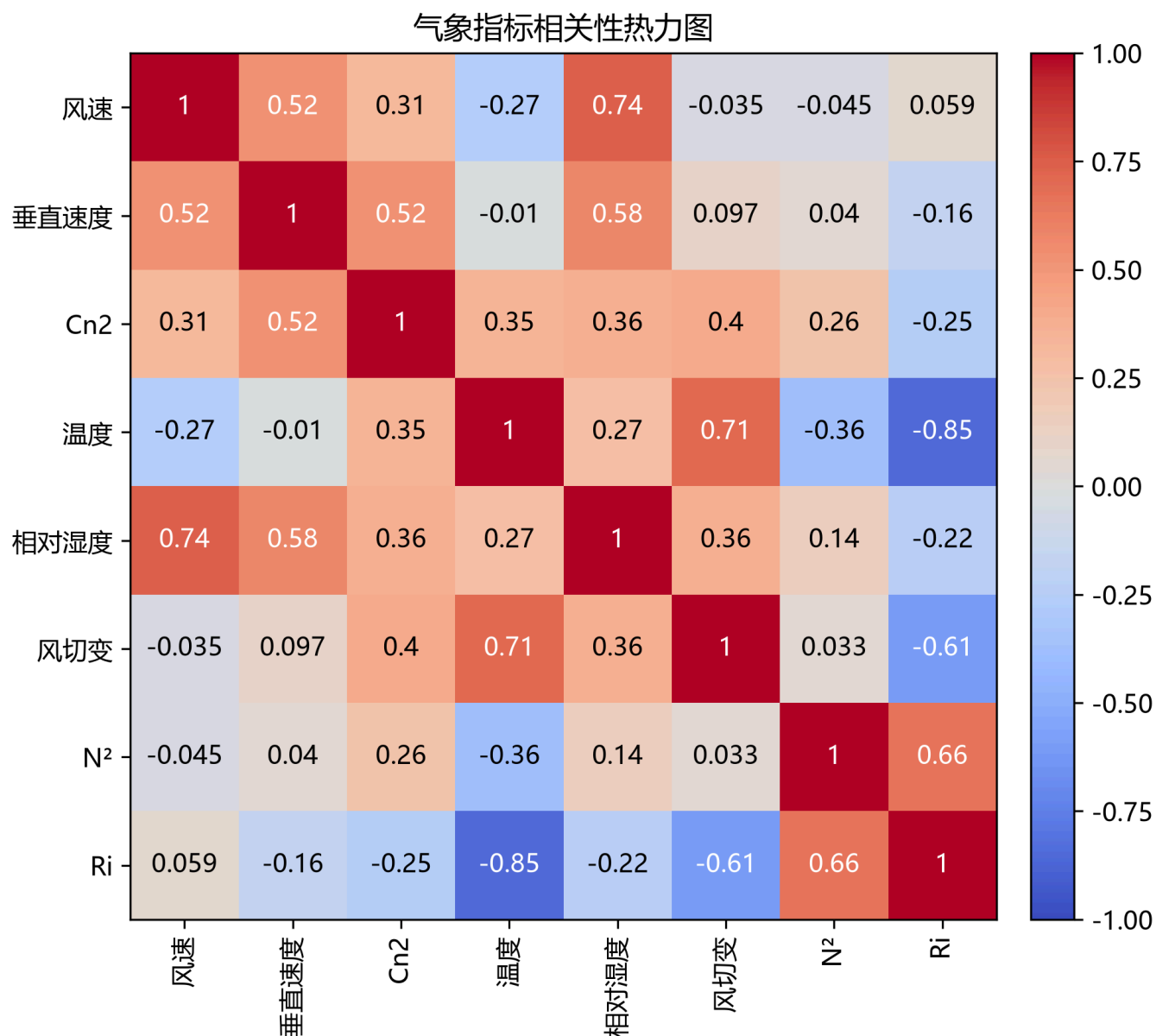
plot_qq(
    fit_data,
    value="residual",
    title="风速拟合残差Q-Q图",
    xlabel="理论正态分位数",
    ylabel="残差分位数",
    colors=get_sci_deep_colors(1),
```

```
        save_path=r"D:\8\Desktop\CMathc\src\utils\output\QQ图-风速拟合残差.png",  
    )
```

若散点大致沿参考直线分布，说明该变量或残差与正态分布较接近；若两端明显偏离，则可能存在厚尾、偏态或异常值。

相关性热力图

```
df =  
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")  
cols = ["wind_speed", "vertical_velocity", "cn2", "temperature",  
        "relative_humidity", "wind_shear", "n2", "ri"]  
labels = ["风速", "垂直速度", "Cn2", "温度", "相对湿度", "风切变", "N²", "Ri"]  
  
plot_corr_heatmap(  
    df,  
    cols=cols,  
    labels=labels,  
    title="气象指标相关性热力图",  
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\相关性热力图-气象指标相关性.png",  
)
```



不确定性

置信区间带图

`plot_band` 用于绘制中心曲线及其上下界阴影带，适合表示“均值 ± 标准差”、置信区间或预测区间。下面按高度统计 A 站风速均值和标准差：

```
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")

band_data = df[
    (df["station"] == "a")
    & (df["height"] <= 1500)
].groupby("height", as_index=False)["wind_speed"].agg(["mean",
"std"]).reset_index()

band_data["lower"] = band_data["mean"] - band_data["std"]
band_data["upper"] = band_data["mean"] + band_data["std"]
```

```
plot_band(
    band_data,
    x="height",
    y="mean",
    lower="lower",
    upper="upper",
    title="A站风速均值及波动范围",
    xlabel="高度 (m)",
    ylabel="风速 (m/s)",
    colors=get_sci_deep_colors(1),
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\置信区间带图-A站风速.png",
)
```

适用场景：多次实验均值与波动、模型预测置信区间、参数敏感性结果等。

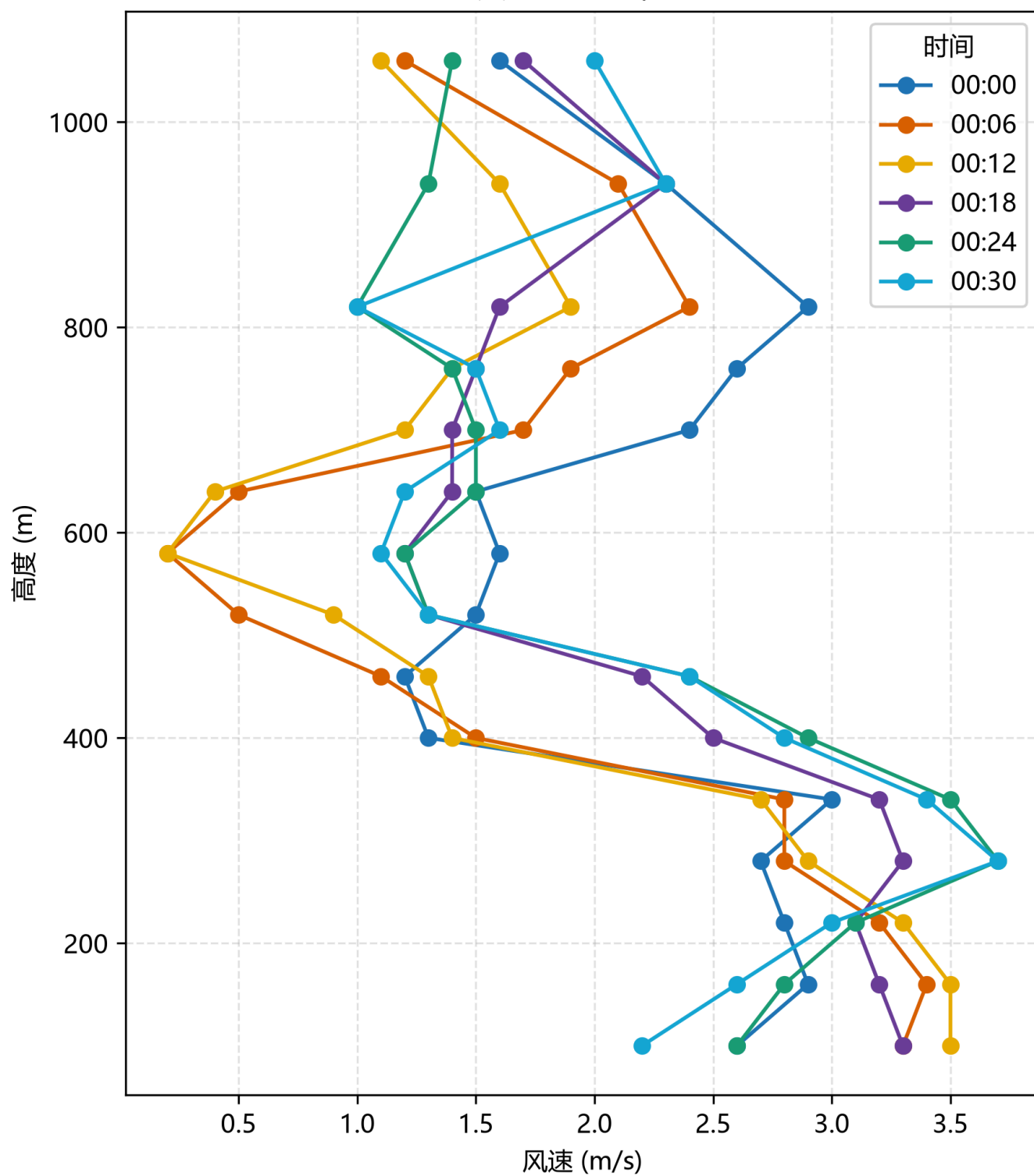
空间与场

廓线图

```
df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")
df["time"] = pd.to_datetime(df["time"])
df["time_label"] = df["time"].dt.strftime("%H:%M")
df_plot = df[(df["station"] == "a") & (df["height"] >= 100) & (df["height"] <=
1100)].copy()

plot_profile(
    df_plot,
    x="wind_speed",
    height="height",
    group="time_label",
    legend="时间",
    title="风速垂直分布图",
    xlabel="风速 (m/s)",
    ylabel="高度 (m)",
    colors=get_sci_deep_colors(df_plot["time_label"].nunique()),
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\廓线图-风速垂直分布.png",
)
```

风速垂直分布图



剖面图

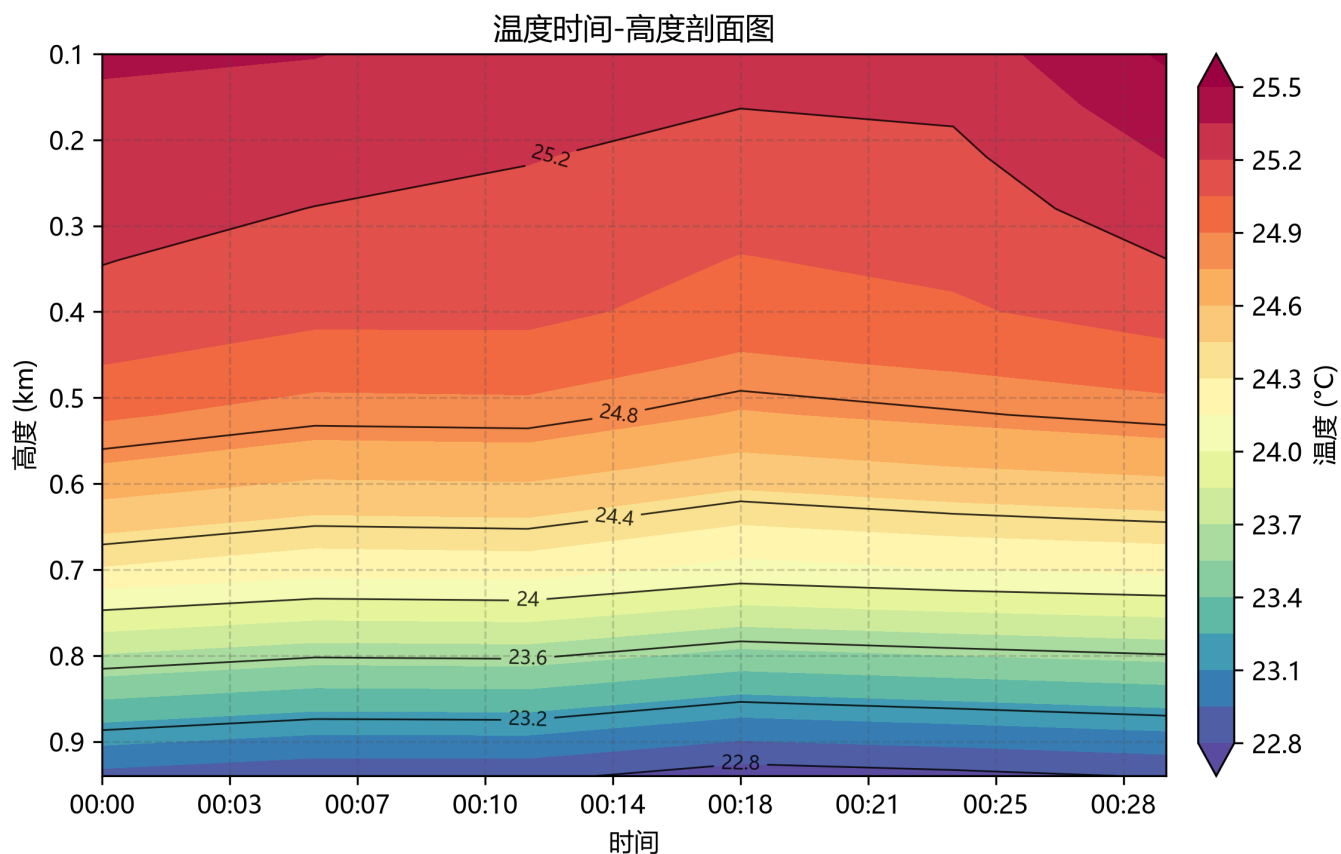
```
df = pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset.csv")
df_plot = df[(df["station"] == "a") & (df["height"] >= 100) & (df["height"] <= 1000)].copy()

plot_section(
    df_plot,
    time="time",
```

```

height="height",
value="temperature",
title="温度时间-高度剖面图",
xlabel="时间",
ylabel="高度 (km)",
colorbar_label="温度 (°C)",
cmap="Spectral_r",
save_path=r"D:\8\Desktop\CMathc\src\utils\output\剖面图-温度时间高度分布.png",
)

```



热力图

```

df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")
df["time"] = pd.to_datetime(df["time"])
df["time_label"] = df["time"].dt.strftime("%H:%M")
df_plot = df[(df["station"] == "a") & (df["height"] >= 100) & (df["height"] <=
1100)].copy()
df_plot["height_label"] = df_plot["height"].astype(int).astype(str)
df_plot["risk"] = df_plot[["cn2",
"wind_shear"]].rank(pct=True).mean(axis=1).fillna(0)

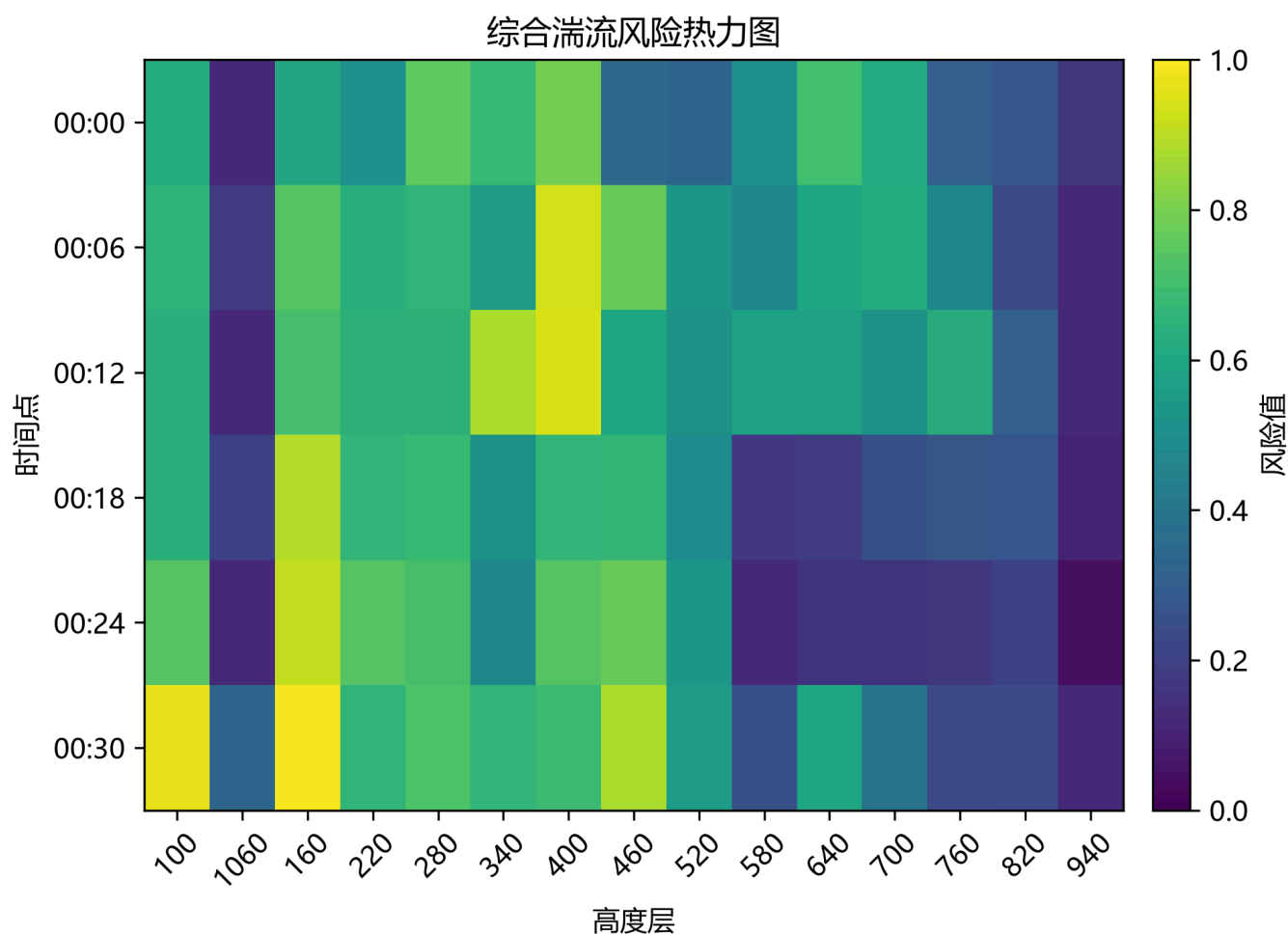
plot_heatmap(
    df_plot,
    x="height_label",
    y="time_label",
    value="risk",

```

```

title="综合湍流风险热力图",
xlabel="高度层",
ylabel="时间点",
colorbar_label="风险值",
cmap="viridis",
vmin=0,
vmax=1,
save_path=r"D:\8\Desktop\CMathc\src\utils\output\热力图-综合湍流风险.png",
)

```



等高线图

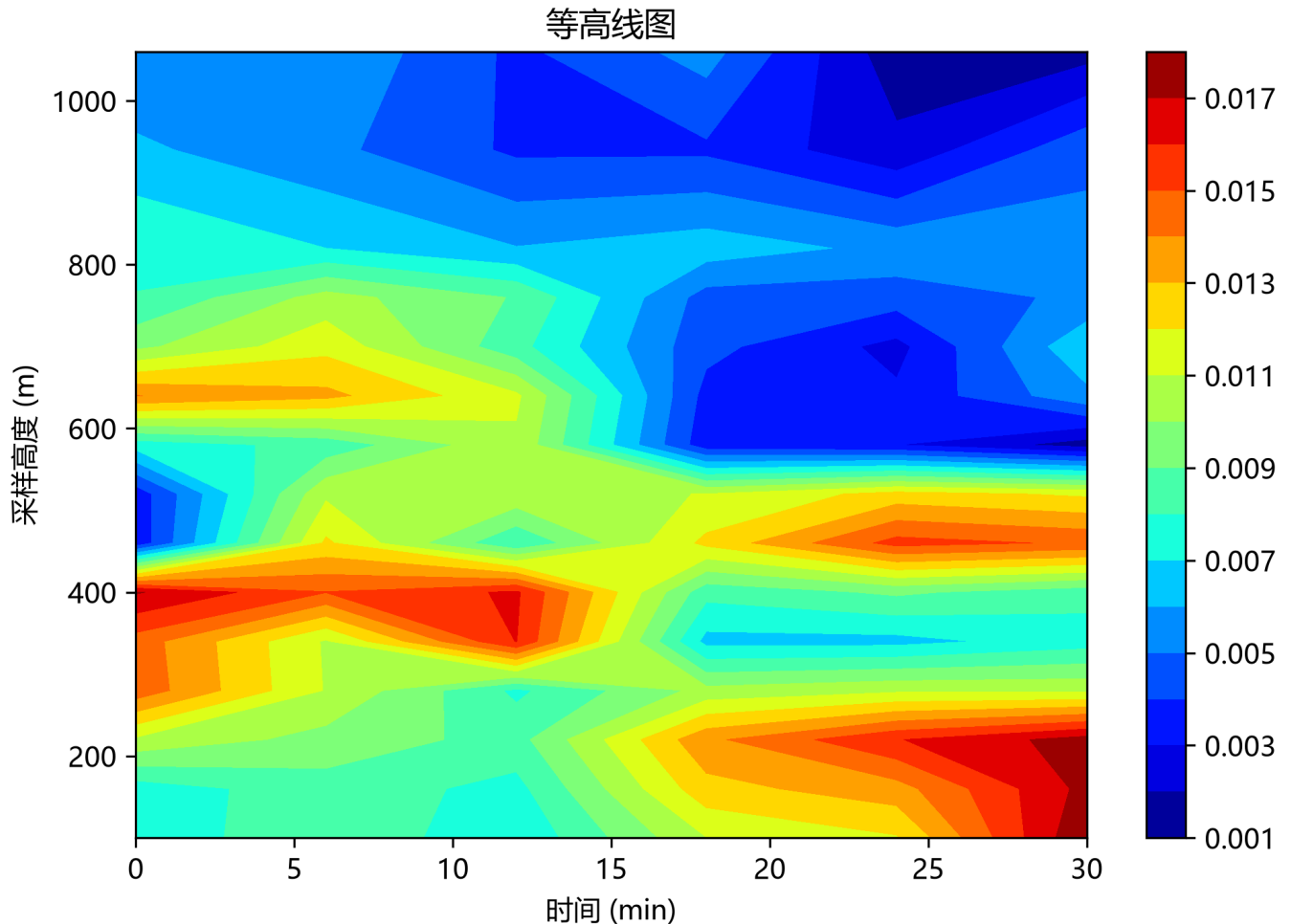
```

df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")
df["time"] = pd.to_datetime(df["time"])
df_plot = df[(df["station"] == "a") & (df["height"] >= 100) & (df["height"] <=
1100)].copy()

plot_contour(
    df_plot,
    x="time",
    y="height",
    value="wind_shear",
    title="等高线图",
)

```

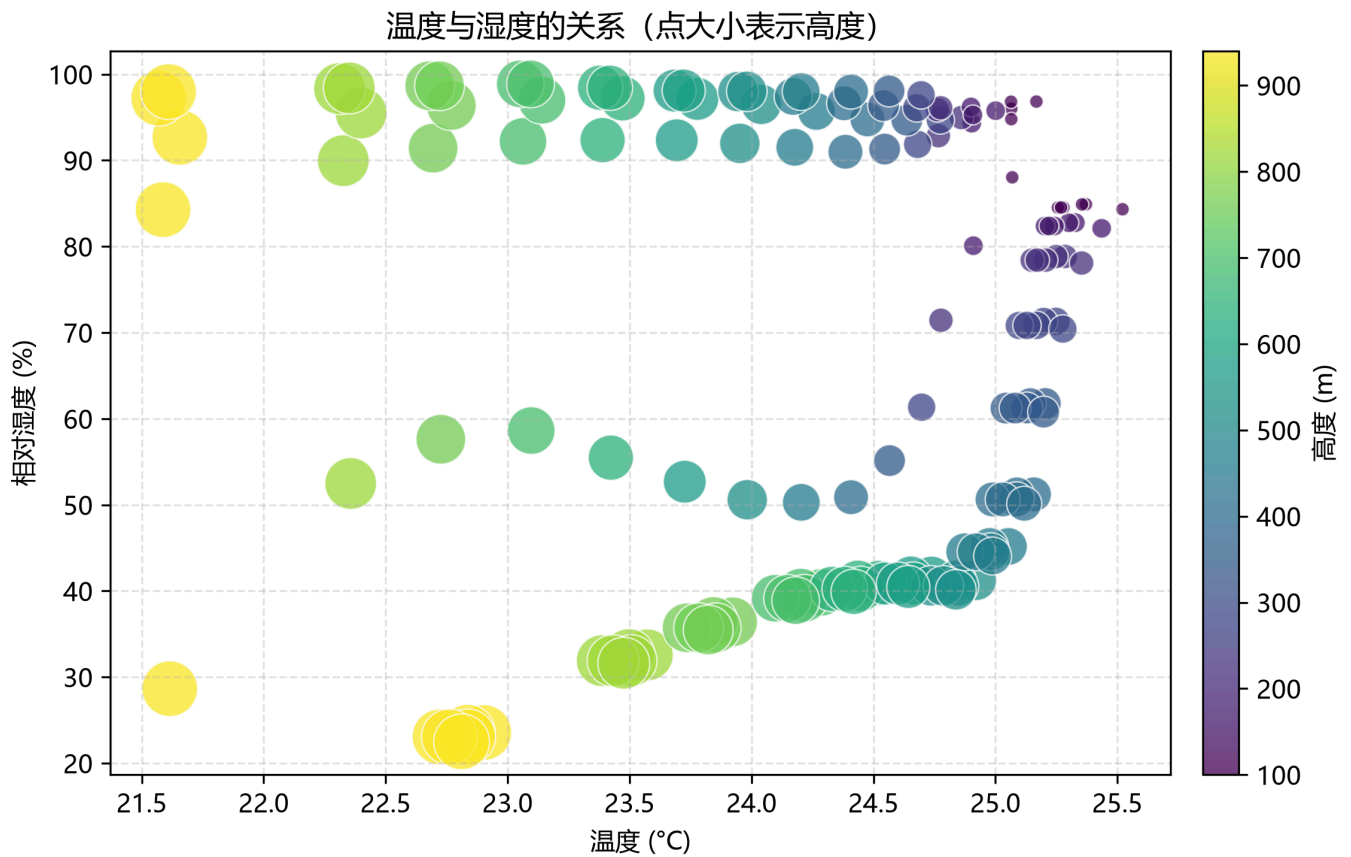
```
xlabel="时间 (min)",  
ylabel="采样高度 (m)",  
cmap="jet",  
save_path=r"D:\8\Desktop\CMathc\src\utils\output\等高线图-风速切变分布.png",  
)
```



气泡图

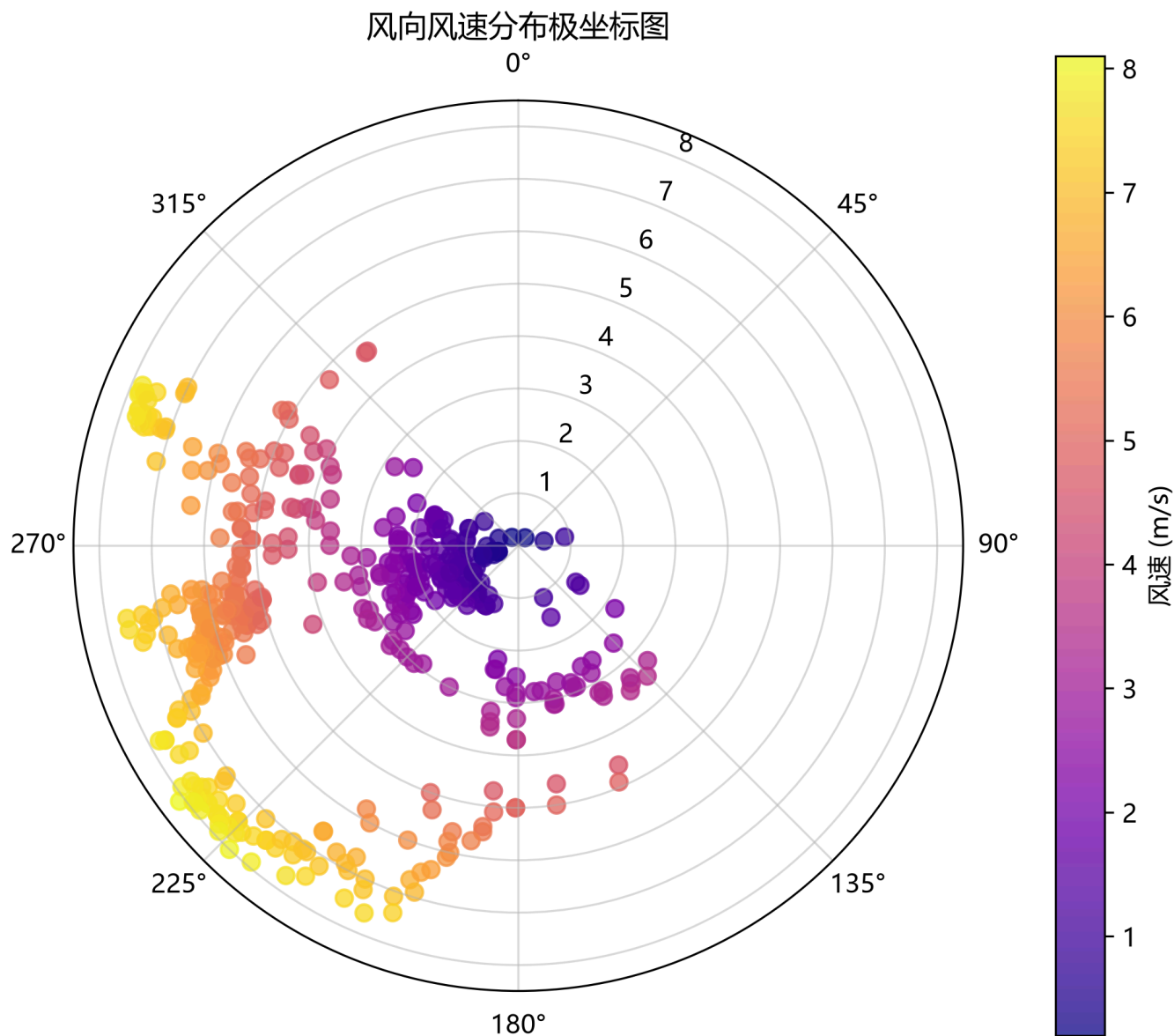
```
df =  
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")  
df_plot = df[(df["height"] >= 100) & (df["height"] <= 1000)].copy()  
  
plot_bubble(  
    df_plot,  
    x="temperature",  
    y="relative_humidity",  
    size="height",  
    color="height",  
    title="温度与湿度的关系（点大小表示高度）",  
    xlabel="温度 (°C)",  
    ylabel="相对湿度 (%)",  
    colorbar_label="高度 (m)",  
    cmap="viridis",  
)
```

```
save_path=r"D:\8\Desktop\CMathc\src\utils\output\气泡图-温度湿度关系.png",  
)
```



极坐标图

```
df =  
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")  
  
plot_wind_polar(  
    df,  
    direction="wind_direction",  
    speed="wind_speed",  
    title="风向风速分布极坐标图",  
    colorbar_label="风速 (m/s)",  
    cmap="plasma",  
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\极坐标图-风向风速分布.png",  
)
```

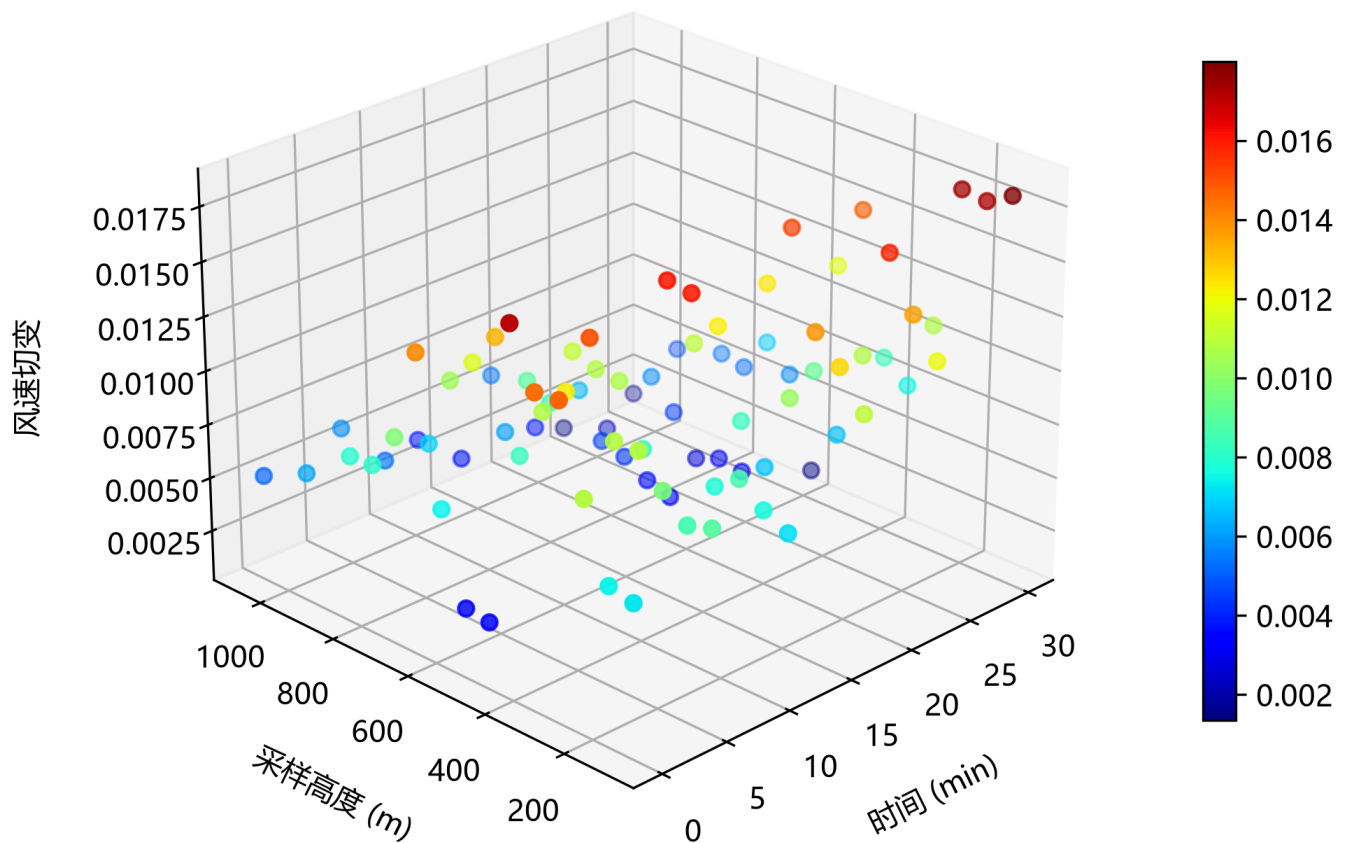


三维散点图

```
df =  
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")  
df["time"] = pd.to_datetime(df["time"])  
df_plot = df[(df["station"] == "a") & (df["height"] >= 100) & (df["height"] <=  
1100)].copy()  
  
plot_3d_scatter(  
    df_plot,  
    x="time",  
    y="height",  
    z="wind_shear",  
    color="wind_shear",  
    title="三维散点图",  
    xlabel="时间 (min)",  
    ylabel="采样高度 (m)",  
    zlabel="风速切变",  
    colorbar_label="风速切变",
```

```
cmap="jet",  
save_path=r"D:\8\Desktop\CMathc\src\utils\output\三维散点图-风速切变分布.png",  
)
```

三维散点图



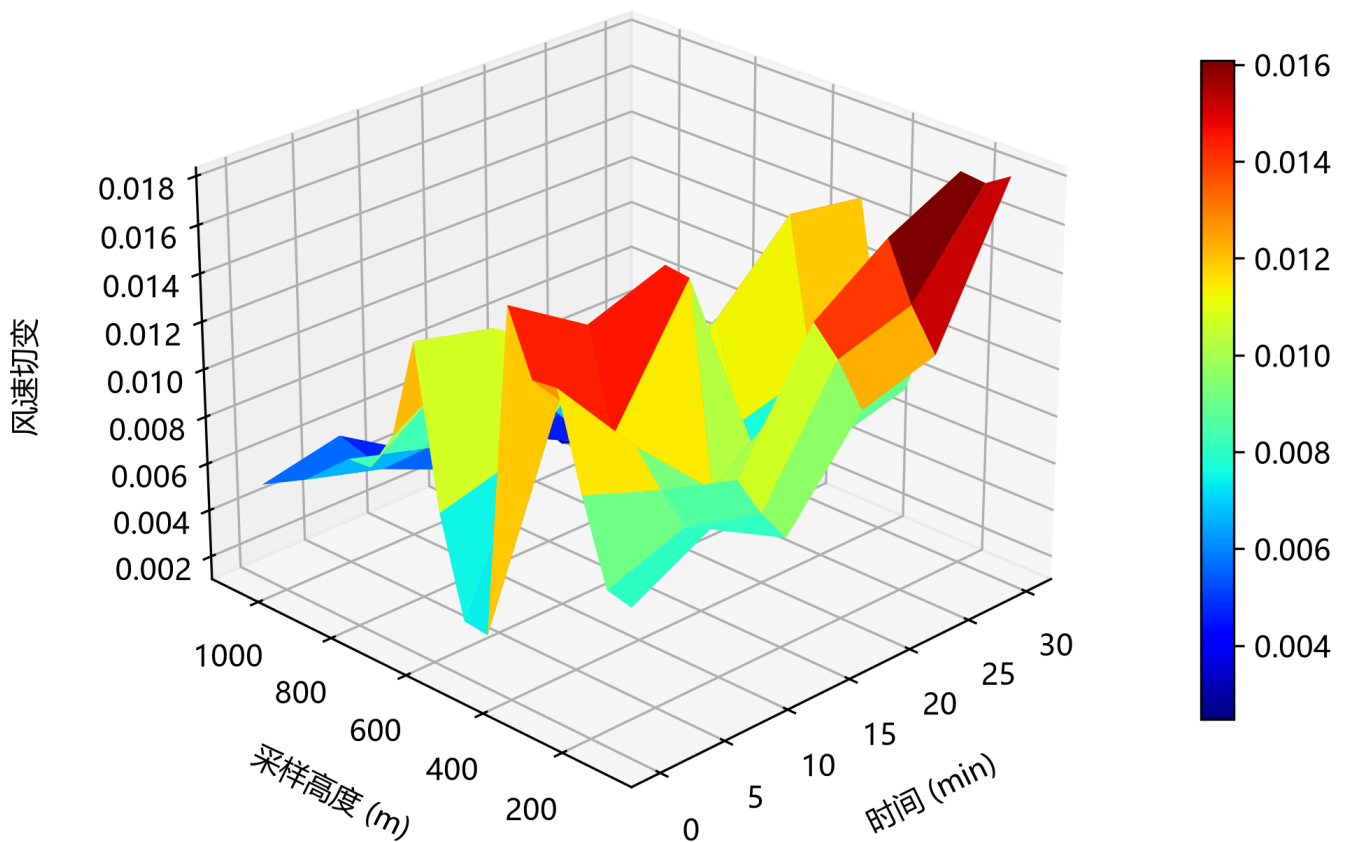
若三维坐标和颜色表示不同变量，例如“经度-纬度-高度”三维散点，颜色表示湍流指标，可写为：

```
plot_3d_scatter(  
    df,  
    x="lon",  
    y="lat",  
    z="height",  
    color="turbulence_index",  
    xlabel="经度",  
    ylabel="纬度",  
    zlabel="高度 (m)",  
    colorbar_label="湍流指标",  
)
```

三维曲面图

```
df =  
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")  
df["time"] = pd.to_datetime(df["time"])  
df_plot = df[(df["station"] == "a") & (df["height"] >= 100) & (df["height"] <=  
1100)].copy()  
  
plot_3d_surface(  
    df_plot,  
    x="time",  
    y="height",  
    value="wind_shear",  
    title="三维曲面图",  
    xlabel="时间 (min)",  
    ylabel="采样高度 (m)",  
    zlabel="风速切变",  
    cmap="jet",  
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\三维曲面图-风速切变分布.png",  
)
```

三维曲面图



三维网格图

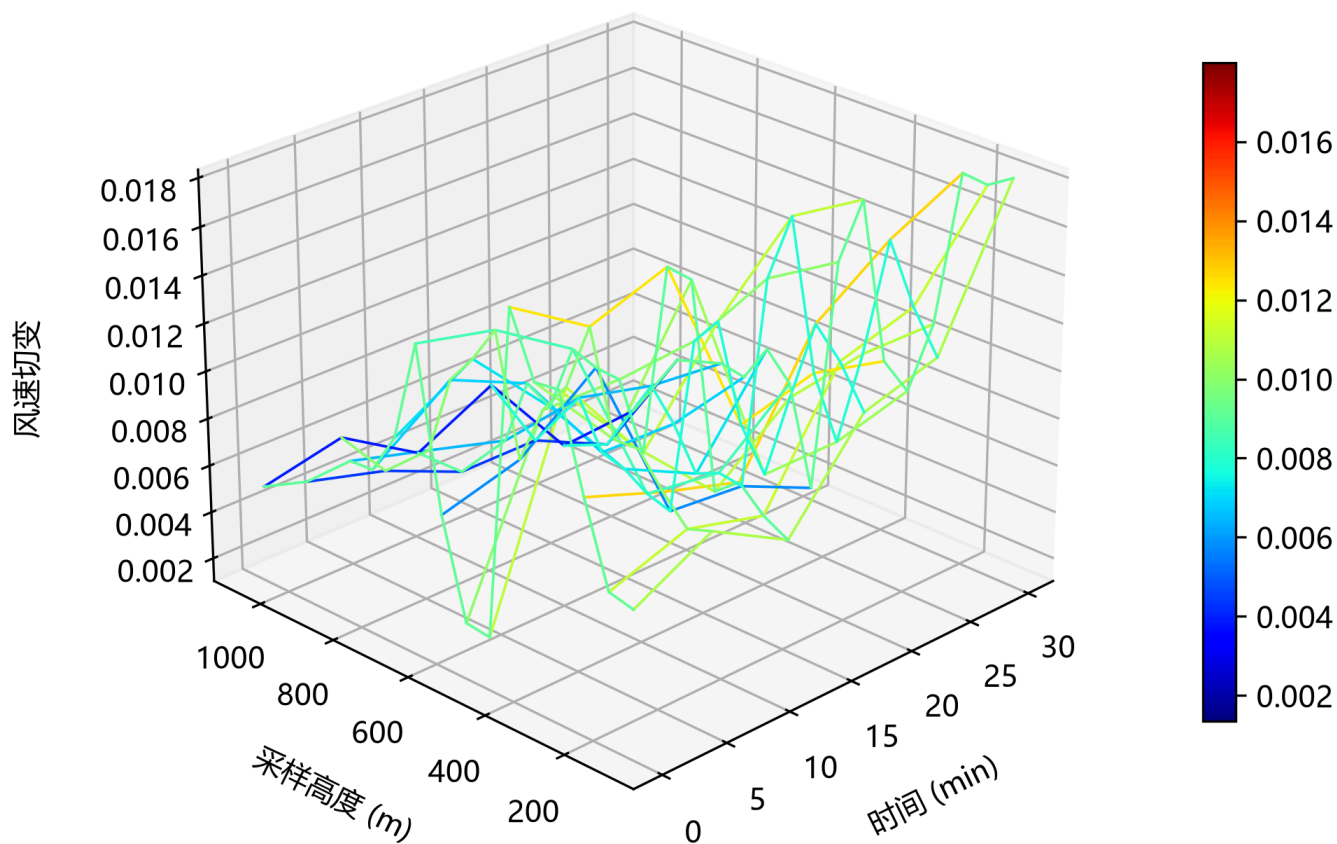
```

df =
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")
df["time"] = pd.to_datetime(df["time"])
df_plot = df[(df["station"] == "a") & (df["height"] >= 100) & (df["height"] <=
1100)].copy()

plot_3d_wireframe(
    df_plot,
    x="time",
    y="height",
    value="wind_shear",
    title="三维网格图",
    xlabel="时间 (min)",
    ylabel="采样高度 (m)",
    zlabel="风速切变",
    cmap="jet",
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\三维网格图-风速切变分布.png",
)

```

三维网格图



GIS 多图层组合图

`plot_gis_layers` 用于递归读取目录中的多个 Shapefile，并将行政区划、河流、湖泊等 GIS 图层叠加到同一个坐标轴中。适合比赛中快速查看随题提供的地理信息数据。

```
gis_root = r"D:\8\Desktop\CMathc\data\D题\题目数据及检验\随题数据\地理信息数据"

styles = {
    "省": {
        "facecolor": "none",
        "edgecolor": "black",
        "linewidth": 1.2,
    },
    "市": {
        "facecolor": "none",
        "edgecolor": "gray",
        "linewidth": 0.7,
    },
    "县": {
        "facecolor": "none",
        "edgecolor": "lightgray",
        "linewidth": 0.4,
    },
    "江苏湖泊水库": {
        "facecolor": "lightskyblue",
        "edgecolor": "dodgerblue",
        "alpha": 0.6,
    },
    "江苏长江等大型河流_面": {
        "facecolor": "skyblue",
        "edgecolor": "dodgerblue",
        "alpha": 0.7,
    },
}

plot_gis_layers(
    data_root=gis_root,
    title="地理信息组合图（江苏区域）",
    styles=styles,
    extent=(116, 122.3, 30.4, 35.5),
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\地理信息组合图.png",
)
```

若需要同时查看每个 .shp 图层的内容，可以设置：

```
plot_gis_layers(
    data_root=gis_root,
    preview_dir=r"D:\8\Desktop\CMathc\src\utils\output\GIS图层预览",
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\地理信息组合图.png",
)
```

这样会额外输出各图层的单独预览图，适合比赛中第一次拿到陌生 GIS 数据时快速检查。

地图热力图

`plot_map_heatmap` 用于展示经纬度空间上的连续风险或强度分布。若已有 Shapefile 地理数据，推荐先调用 `plot_gis_layers` 绘制 GIS 底图，再将热力图叠加到同一个 `ax` 上。

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

gis_root = r"D:\8\Desktop\CMathc\data\D题\题目数据及检验\随题数据\地理信息数据"

# 示例空间风险数据
np.random.seed(1)
df_map = pd.DataFrame({
    "lon": np.r_[
        np.random.normal(118.8, 0.25, 100),
        np.random.normal(120.5, 0.25, 80),
        np.random.normal(119.5, 0.20, 60),
    ],
    "lat": np.r_[
        np.random.normal(32.1, 0.20, 100),
        np.random.normal(31.6, 0.18, 80),
        np.random.normal(33.2, 0.20, 60),
    ],
    "risk": np.r_[
        np.random.uniform(0.6, 1.0, 100),
        np.random.uniform(0.4, 0.9, 80),
        np.random.uniform(0.3, 0.8, 60),
    ],
})

map_extent = (116, 122.3, 30.4, 35.5)

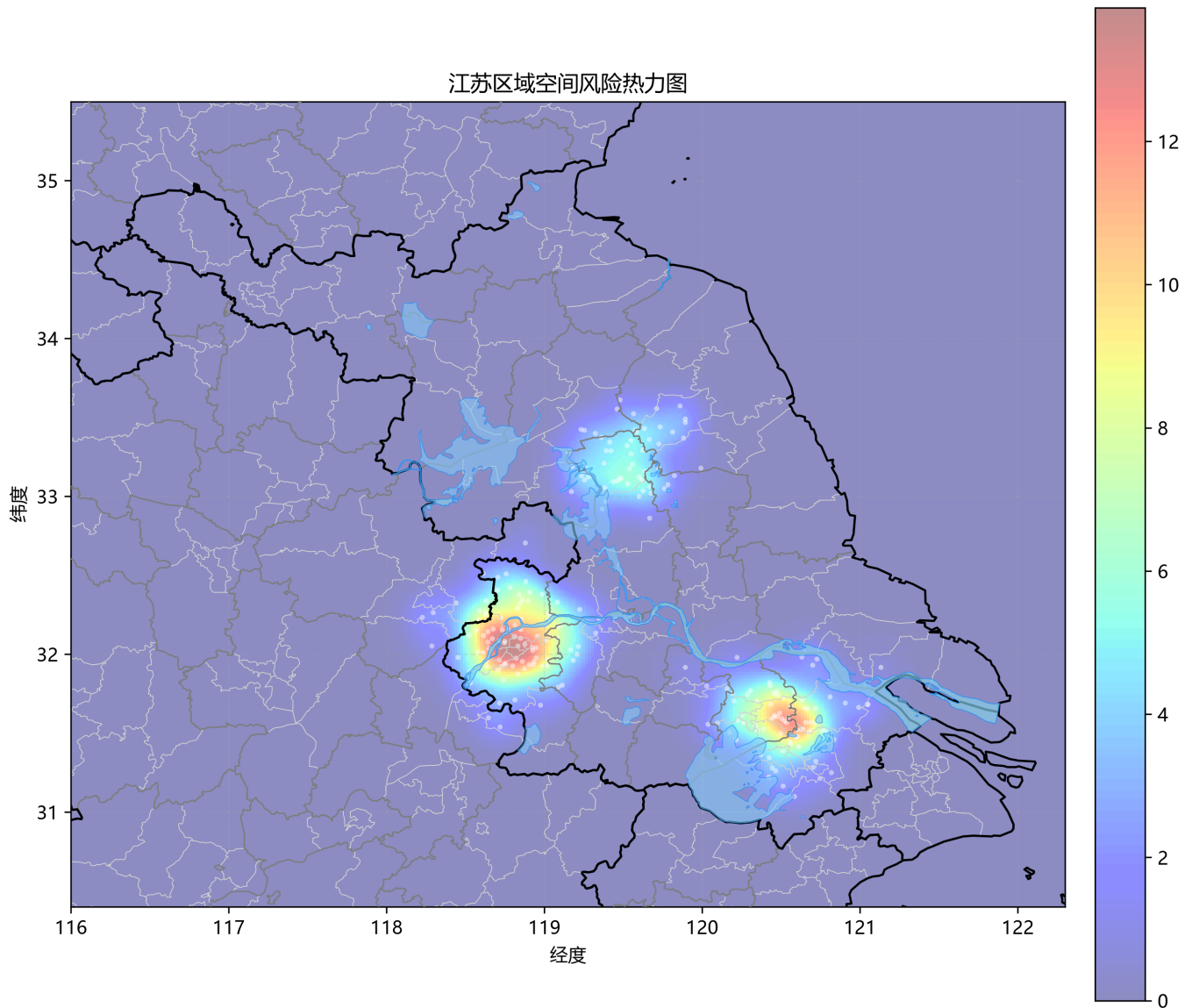
gis_styles = {
    "省": {"facecolor": "none", "edgecolor": "black", "linewidth": 1.2},
    "市": {"facecolor": "none", "edgecolor": "gray", "linewidth": 0.7},
    "县": {"facecolor": "none", "edgecolor": "lightgray", "linewidth": 0.4},
    "江苏湖泊水库": {
        "facecolor": "lightskyblue",
        "edgecolor": "dodgerblue",
        "alpha": 0.6,
    },
    "江苏长江等大型河流_面": {
        "facecolor": "skyblue",
        "edgecolor": "dodgerblue",
        "alpha": 0.7,
    },
}

fig, ax = plt.subplots(figsize=(10, 8))

# 先画 GIS 底图
plot_gis_layers(
    data_root=gis_root,
```

```
        ax=ax,
        extent=map_extent,
        styles=gis_styles,
    )

# 再叠加热力图
plot_map_heatmap(
    df_map,
    x="lon",
    y="lat",
    value="risk",
    ax=ax,
    extent=map_extent,
    title="江苏区域空间风险热力图",
    xlabel="经度",
    ylabel="纬度",
    cmap="jet",
    alpha=0.45,
    radius=0.10,
    grid_size=350,
    save_path=r"D:\8\Desktop\CMathc\src\utils\output\地图热力图-GIS底图.png",
)
```



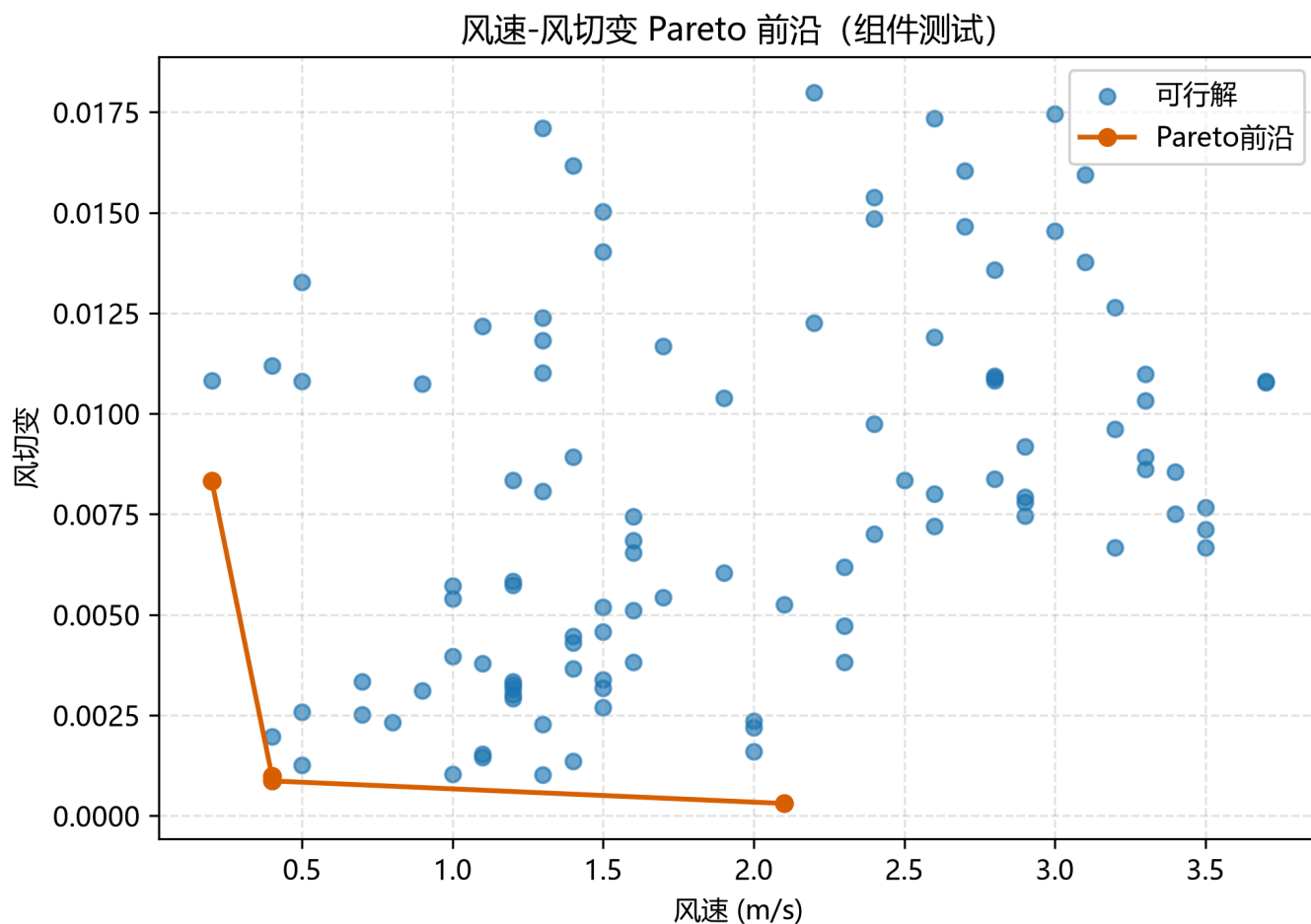
优化

Pareto前沿图

`plot_pareto` 用于双目标优化结果展示。散点表示可行解，连线标出非支配的 Pareto 前沿。下面仅使用现有风速和风切变数据演示组件调用，不表示正式优化模型中的目标函数：

```
df =  
pd.read_csv(r"D:\8\Desktop\CMathc\src\test\q1\output\q1_dataset_features.csv")  
  
pareto_data = df[  
    (df["station"] == "a")  
    & (df["height"] <= 1500)  
][["wind_speed", "wind_shear"]].dropna().copy()  
  
plot_pareto(  
    pareto_data,  
    x="wind_speed",  
    y="wind_shear",  
    minimize_x=True,
```

```
minimize_y=True,  
title="风速-风切变 Pareto 前沿（组件测试）",  
xlabel="风速 (m/s)",  
ylabel="风切变",  
colors=get_sci_deep_colors(2),  
save_path=r"D:\8\Desktop\CMathc\src\utils\output\Pareto前沿图-组件测试.png",  
)
```



`minimize_x=True` 和 `minimize_y=True` 表示两个目标都希望越小越好；若某个目标是最大化，将对应参数改为 `False` 即可。正式建模时应把 `x`、`y` 换成实际的两个优化目标，例如成本-风险、误差-复杂度或收益-能耗。